Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Компьютерные сети»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Анализ трафика компьютерных сетей утилитой Wireshark»

Выполнил:
Шибаев Иван Дмитриевич, студент группы N3352

_____
(подпись)

Проверил:
Есипов Дмитрий Андреевич

_____
(подпись)

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение 3

1 Экспериментальная часть 4

1.1 Методика и стенд 4

1.2 Анализ трафика ping 7

1.3 Анализ traceroute 13

1.4 Анализ HTTP 15

1.5 Анализ DNS и nslookup 18

1.6 Анализ ARP 22

1.7 Анализ FTP 23

1.8 Анализ DHCP 26

1.9 Анализ трафика Яндекс Телемоста 29

Заключение 38

Список использованных источников 39

ВВЕДЕНИЕ

Цель работы — изучить структуру протокольных блоков данных и порядок сетевого обмена с помощью Wireshark. Работа соответствует заданию 3 методического пособия [1]. Исследуются ICMP и IPv4 на примерах ping и traceroute, обычный и условный HTTP GET, разрешение имён DNS, определение адреса следующего узла через ARP, команды nslookup, передача файла по FTP и назначение адреса по DHCP.

Для каждого опыта сохранены реальные сетевые пакеты и журналы команд; измеренные величины извлекаются из трасс повторно.

Для достижения цели поставлены следующие задачи:

- измерить число IPv4-фрагментов при изменении объёма данных ICMP и проследить работу TTL на маршруте с несколькими переходами;
- сопоставить обычный и условный GET, поля запросов и ответов DNS, канальные и межсетевые адреса;
- исследовать управляющее соединение и передачу файла FTP, а также два цикла выдачи адреса DHCP;
- сравнить наблюдаемые аудио- и видеопотоки Телемоста и установить границы их распознавания по сетевому дампу.

Внешним объектом исследования служит ИСУ ИТМО. Для управления условиями части опытов используется контейнерный стенд. В отчёте отдельно указаны результаты внешних измерений, локальных экспериментов и наблюдений в интерфейсе приложений; это позволяет проверить, какие выводы подтверждаются полями пакетов, а какие требуют сведений приложения.

1 Экспериментальная часть

1.1 Методика и стенд

1.1.1 Выбранный сайт и границы наблюдения

Для внешних проб выбран сайт ИСУ ИТМО — isu.ifmo.ru, указанный студентом. Проверки выполнены на macOS средствами dumpcap, ping, traceroute и nslookup; числовые поля извлечены TShark 4.6.9. Скриншоты анализа сохранённых трасс сделаны в официальной Wireshark 4.4.19: установленная версия 4.6.9 аварийно завершалась при работе через интерфейс доступности Qt.

В пробах 2 ноября IPv4-адрес сайта равнялся 77.234.212.21, DNS-резолвер VPN — 172.19.0.2; маршрут по умолчанию проходил через utun4. 3 ноября в открытой браузерной сессии загружена страница профиля ИСУ с заголовком «Шибаев Иван Дмитриевич — ИСУ ИТМО». В содержимом указаны ФИО студента, третий курс и группа N3352.

Это подтверждает выбор сайта по варианту. ИСУ использует HTTPS, поэтому сведения профиля подтверждены интерфейсом, а не открытыми полями сетевого дампа. Идентификаторы браузерной сессии и номер студента в материалы работы не включены. При работе через туннель наблюдалось следующее поведение сети. ICMP-вариант внешней трассировки показал ответ целевого адреса уже при TTL, равном единице; UDP-вариант не получил ответов за шесть попыток.

Для больших Echo Request утилита ping сообщала об ошибке «Message too long». Эти результаты сохранены как свойства доступного пути и его обработки. По ним нельзя восстановить физическое число маршрутизаторов до ИСУ или надёжно измерить обычную фрагментацию IPv4. Запрос HEAD к корню сайта без браузерной сессии получил HTTP/2 403. Он не подтверждает загрузку личного кабинета. Содержимое авторизованной страницы ИСУ защищено HTTPS; ключи TLS и идентификатор пользовательской сессии в материалы работы не включены.

Для наблюдения открытых заголовков и управляемого маршрута использован отдельный контейнерный стенд. Его обмены действительно передавались между сетевыми стеками Linux и записывались в pcap. HTTP

и FTP на стенде исследованы клиентом curl; обычный и условный GET дополнительно записаны в Chrome. Очистка системного DNS-кэша macOS в этих HTTP-опытах не выполнялась; отдельный DNS-опыт после очистки кэша проведён 4 ноября и описан ниже.

1.1.2 Состав стенда

Адреса узлов приведены в таблице 1. Путь от клиента к серверу проходит через два маршрутизатора и три Ethernet-сегмента. На интерфейсах использован MTU 1500 байт. Отдельный клиент DHCP находится в четвёртом сегменте; при опыте адрес удалялся только в его контейнере.

Таблица 1 — Адреса контейнерного стенда

Узел	**IPv4-адрес**	**Назначение**
Клиент	172.28.20.10/24	ping, traceroute, HTTP, DNS, FTP
R1	172.28.20.254/24; 172.28.21.10/24	Первый маршрутизатор
R2	172.28.21.20/24; 172.28.22.254/24	Второй маршрутизатор
Сервер	172.28.22.10/24; 172.28.23.2/24	HTTP, FTP, DNS, DHCP
Клиент DHCP	172.28.23.101/24 после DHCP	Получение и освобождение аренды

HTTP обслуживает SimpleHTTPServer из Python 3.12.12 на порту 8000; FTP — pyftpdlib 2.1.0 на порту 21, DNS и DHCP — dnsmasq 2.90. Учебный файл index.html содержит ФИО студента, указание на локальное назначение страницы и занимает 476 байт. Учебное имя shibaev.test разрешается DNS-сервером стенда в 172.28.22.10.

На рисунке 1 показаны три сегмента основного маршрута и отдельная сеть DHCP. Стрелки обозначают соседство узлов, а не дополнительный измеренный трафик.

Клиент 172.28.20.10 ⟶ R1 172.28.20.254
R1 172.28.21.10 ⟶ R2 172.28.21.20
R2 172.28.22.254 ⟶ сервер 172.28.22.10
Сервер 172.28.23.2 ↔ клиент DHCP 172.28.23.101
Рисунок 1 — Адреса и соединения узлов учебного стенда

1.1.3 Инкапсуляция и назначение заголовков

Модель OSI служит способом разделить функции сетевого взаимодействия; реальные протоколы TCP/IP не обязаны иметь по отдельному заголовку для каждого из семи уровней. Для сохранённых обменов порядок инкапсуляции показан в таблице 2. Обозначение «данные» в этой таблице относится к полезной нагрузке соответствующего протокола.

Таблица 2 — Инкапсуляция наблюдаемых пакетов

Обмен	**Порядок протокольных блоков**
ping	Ethernet II ⟶ IPv4 ⟶ ICMP Echo ⟶ данные
Ошибка ICMP	Ethernet II ⟶ IPv4 ⟶ ICMP error ⟶ исходный IPv4 и начало его нагрузки
HTTP, FTP	Ethernet II ⟶ IPv4 ⟶ TCP ⟶ прикладные команды или содержимое файла
DNS	Ethernet II ⟶ IPv4 ⟶ UDP ⟶ DNS
ARP	Ethernet II ⟶ ARP; заголовков IP и TCP/UDP нет
DHCP	Ethernet II ⟶ IPv4 ⟶ UDP ⟶ сообщение формата BOOTP и DHCP-опции

Ethernet II использует MAC-адреса для доставки внутри текущего канального сегмента и EtherType для указания вложенного протокола. IPv4 содержит межсетевые адреса, полную длину, TTL, идентификатор, признаки фрагментации и номер следующего протокола. TCP добавляет порты, последовательные номера, подтверждения, окно и управляющие флаги; UDP — порты, длину и контрольную сумму. Прикладной заголовок задаёт смысл операции: запрос ресурса, вопрос к DNS, команду FTP или тип сообщения DHCP.

В обратном направлении получатель разбирает эти оболочки в обратном порядке. Смена запроса на ответ меняет роли источника и

назначения; при маршрутизации между сегментами канальные адреса заменяются, а конечные IPv4-адреса сохраняются при отсутствии NAT. Физические символы и радиосигнал данным захватом не измерялись.

1.2 Анализ трафика ping

1.2.1 Команды и структура Echo

Для локального измерения фрагментации выполнены одиночные Echo Request с размером данных от 100 до 10000 байт. Дополнительно выбраны значения 1472 и 1473 байта, позволяющие проверить границу MTU. Пример команды:

```
ping -n -c 1 -W 2 -M dont -s 2000 \
  172.28.22.10
```

В этой реализации ключ «-s» задаёт данные ICMP, «-M dont» разрешает фрагментацию. Для анализа всех фрагментов удобен фильтр:

```
ip.src == 172.28.20.10 && ip.proto == 1
```

Один фильтр «icmp» может быть неудобен для подсчёта: фрагменты с ненулевым смещением не содержат собственного заголовка ICMP. Поэтому фрагменты сопоставлялись по IPv4-источнику, получателю, Protocol и Identification. У Echo Request тип ICMP равен 8, у Echo Reply — 0, Code в обоих случаях равен нулю. Заголовок Echo содержит контрольную сумму, идентификатор и номер последовательности.

Ответ возвращает данные запроса, позволяя сопоставить пару и проверить содержимое [2]. В кадре 1 трассы stand-ping.pcap сообщение со 100 байтами данных имеет IPv4 Total Length 128 байт: 20 байт заголовка IPv4, 8 байт заголовка Echo и 100 байт данных. Длина Ethernet-кадра в захвате — 142 байта; преамбула и межкадровый интервал в неё не входят.

1.2.2 Фрагментация и число фрагментов

При MTU 1500 байт и заголовке IPv4 длиной 20 байт фрагмент может нести до 1480 байт нагрузки. Для всех промежуточных фрагментов её длина должна быть кратна восьми; 1480 удовлетворяет этому условию. Для данных Echo длиной S ожидаемое число IPv4-пакетов в данном опыте равно

$$N = \left\lceil \frac{S + 8}{1480} \right\rceil. \tag{1}$$

Формула (1) учитывает восьмибайтовый заголовок ICMP внутри исходной нагрузки IPv4. Поля Identification, More Fragments (MF) и Fragment Offset позволяют объединить фрагменты; смещение задаётся в единицах по восемь байт [3]. Измеренный подсчёт приведён в таблице 3 и на рисунке 2.

Таблица 3 — Наблюдаемая зависимость фрагментации от размера Echo Request

Данные ICMP, байт	**IPv4-пакетов в запросе**
100	1
1000	1
1472	1
1473	2
2000	2
3000	3
4000	3
5000	4
6000	5
7000	5
8000	6
9000	7
10000	7

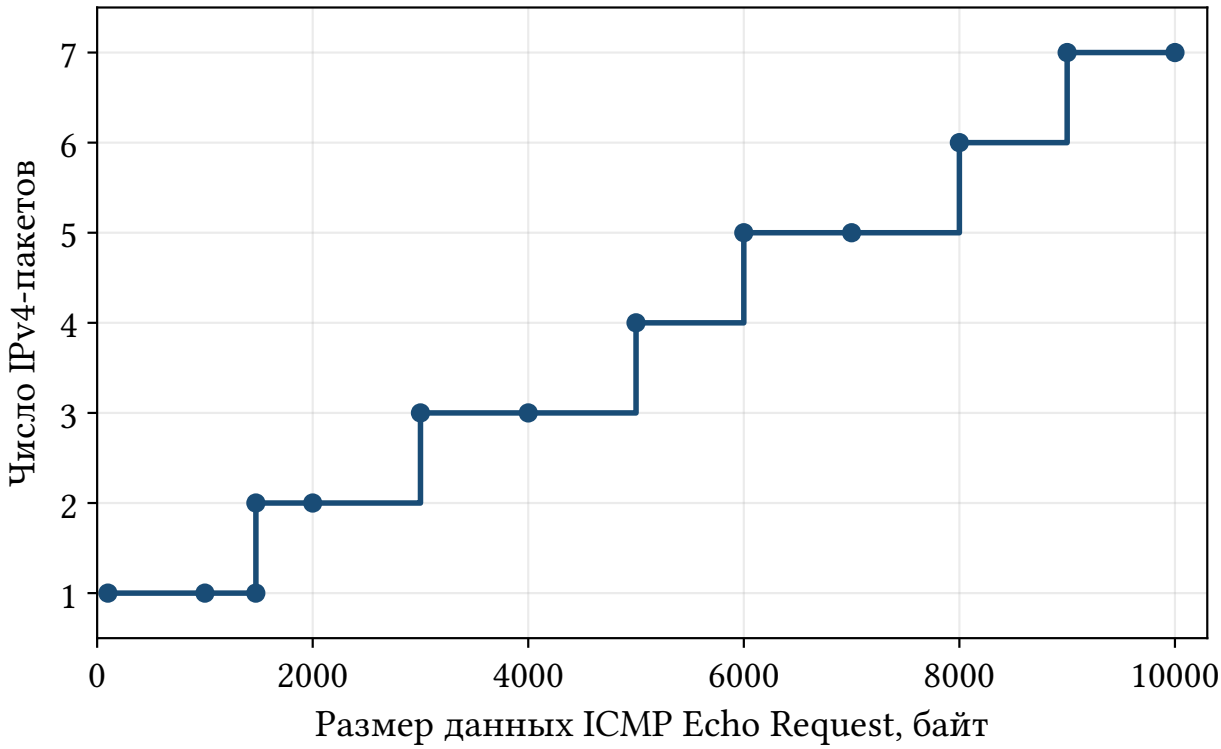


Рисунок 2 — Число IPv4-пакетов для одного Echo Request при MTU 1500 байт

Для S = 2000 байт кадры 11 и 12 имеют общий Identification 0x8b13 и длины IPv4 1500 и 548 байт. У первого фрагмента MF=1 и смещение 0; у последнего MF=0 и смещение 185, то есть 185 × 8 = 1480 байт. Суммарная нагрузка равна 1480 + 528 = 2008 байт. После вычитания заголовка Echo остаются заданные 2000 байт. На рисунке 3 показано разбиение этого запроса по реально захваченным полям.

Каждый фрагмент имеет собственный заголовок IPv4. Заголовок ICMP находится только в начале первой части; второй фрагмент продолжает её данные. Схема показывает состав, размеры блоков на ней не масштабируются по числу байтов.

Кадр 11: ID 0x8b13; MF=1; Offset=0

IPv4 20 байт	ICMP 8 байт + данные 1472 байта Нагрузка фрагмента: 1480 байт

Кадр 12: ID 0x8b13; MF=0; Offset=185

IPv4 20 байт	Продолжение данных: 528 байт Заголовка ICMP здесь нет

Рисунок 3 — Разбор двух IPv4-фрагментов запроса с 2000 байтами данных

Признак MF=1 указывает, что после текущего фрагмента есть продолжение. У последнего MF=0; его ненулевое смещение отличает его от обычного нефрагментированного пакета. Промежуточный фрагмент имеет MF=1 и ненулевое смещение. Для 10000 байт данных запрос разбит на семь фрагментов: шесть длиной IPv4 1500 байт и последний — 1148 байт.

Поля первого и последнего фрагментов в Wireshark сопоставлены на рисунках 4 и 5. GUI показывает смещение последнего фрагмента сразу в байтах — 1480, тогда как поле TShark ip.frag_offset содержит исходное значение 185 в восьмибайтовых единицах. Это два представления одного и того же смещения.

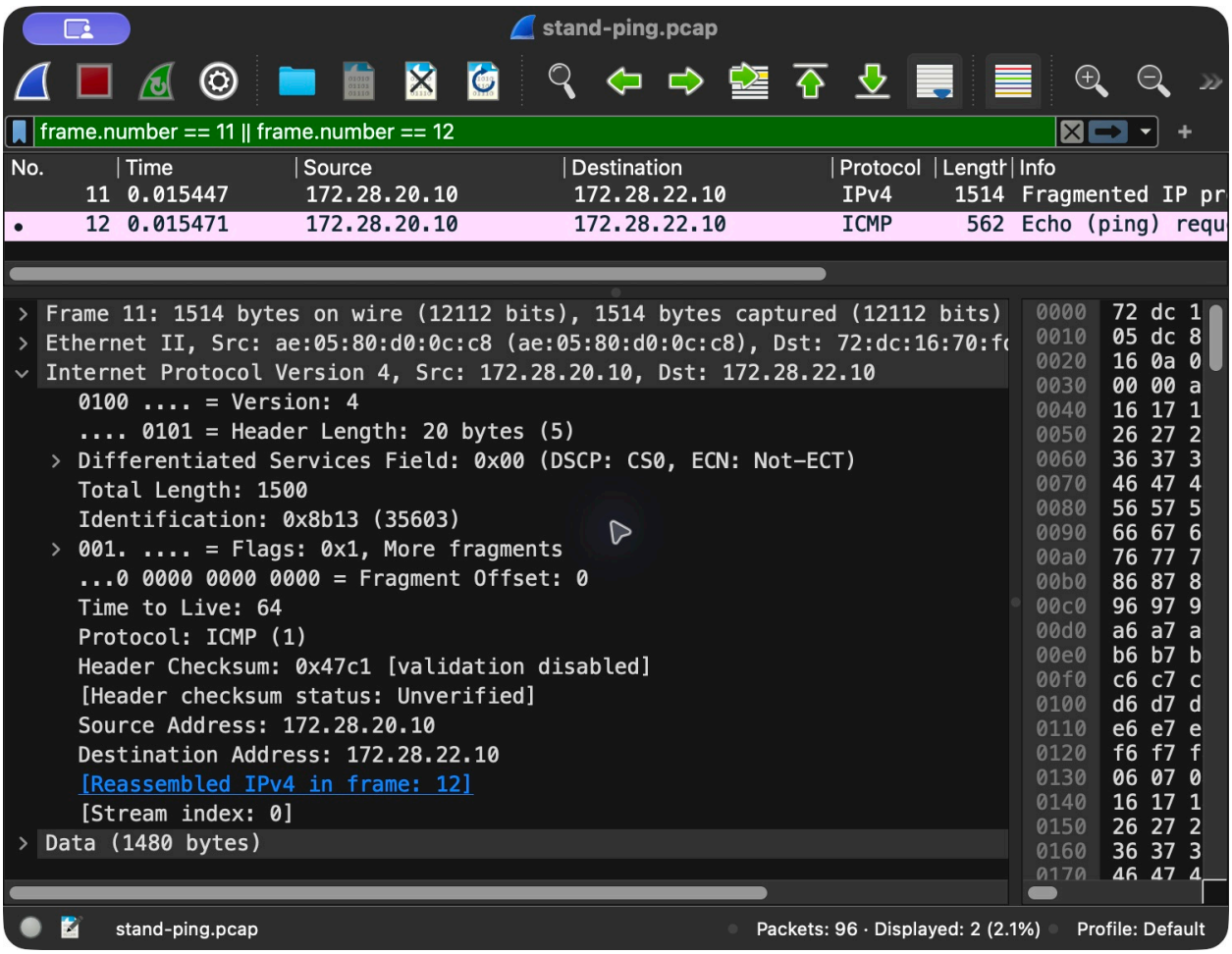


Рисунок 4 — Первый IPv4-фрагмент запроса: кадр 11, Identification 0x8b13

В кадре 12 на рисунке 5 флаг More Fragments снят. Совпадение Identification с первым фрагментом и ненулевой Fragment Offset подтверждают принадлежность к тому же запросу и положение в его конце.

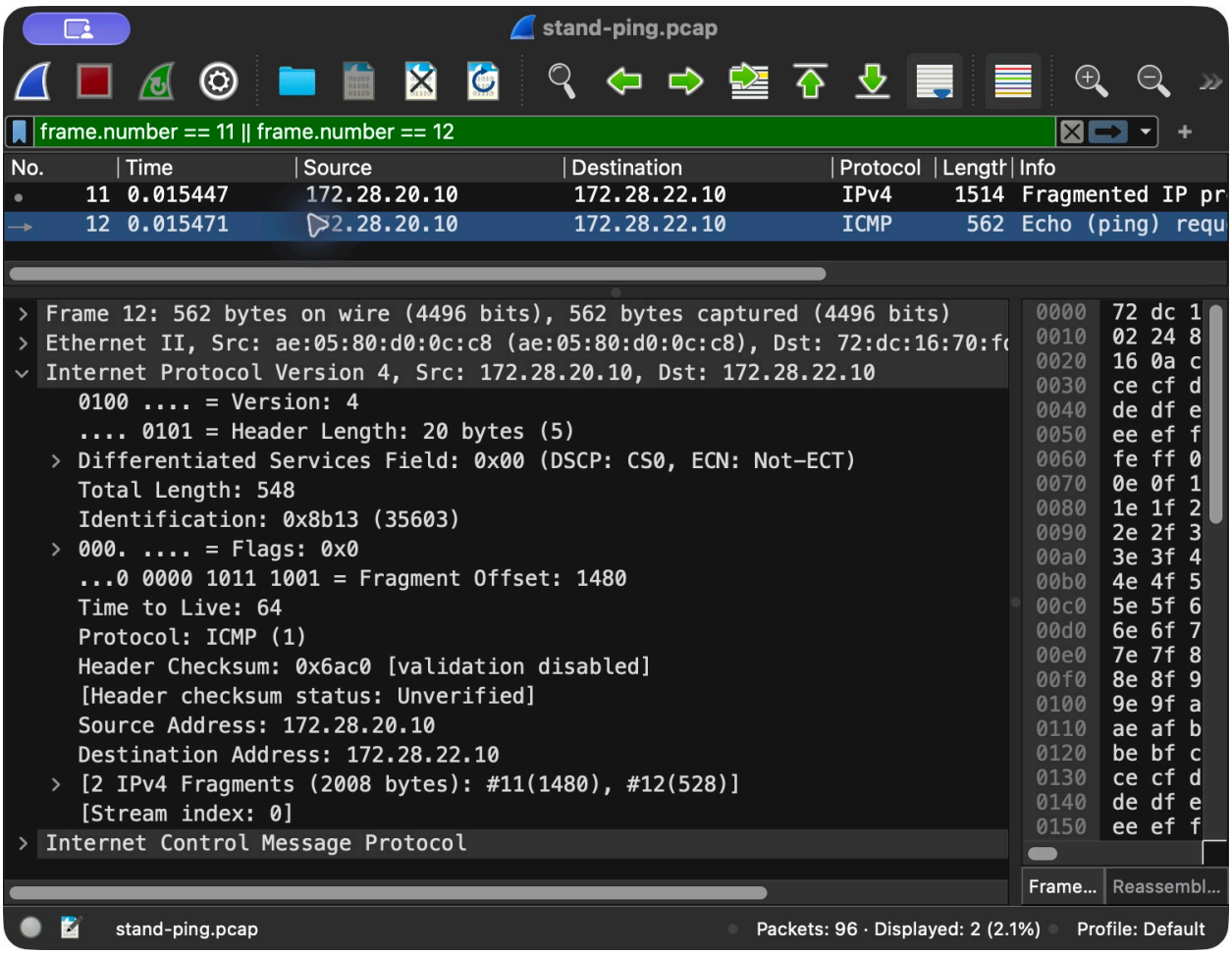


Рисунок 5 — Последний IPv4-фрагмент запроса: кадр 12, смещение 1480 байт

1.2.3 TTL и данные запроса

На Linux ключ «-t» задаёт начальный TTL. В дополнительной команде ниже кадр 95 содержит TTL=7:

```
ping -n -c 1 -W 2 -t 7 -p abcd -s 100 \
  172.28.22.10
```

В macOS аналогичный параметр — «-m», а в Windows — «-i». Параметры разных реализаций нельзя переносить без проверки их справки. TTL в полученном Echo Reply равен 62: это значение относится к новому ответному пакету, а не к оставшемуся TTL запроса. Поле данных зависит от реализации утилиты. В кадре 95 первые 16 байт данных использованы реализацией ping для временной метки, оставшиеся 84 байта содержат повторяющийся шаблон AB CD, выбранный ключом «-p abcd». Поэтому данные Echo не следует описывать как обязательный фиксированный текст или последовательность букв.

В ответе кадра 96 эти данные возвращены.

1.2.4 Результат внешней пробы

Внешние команды направлялись адресу ИСУ. При 1472 байтах данных ответ имел ожидаемую длину; при 1473 байтах утилита вывела предупреждение «wrong total length 1500 instead of 1501». Для больших размеров, включая 10000 байт, отправка завершилась ошибкой «Message too long». Таблица 3 относится к управляемому стенду, где сохранены полноценные запросы и ответы, а не к этим неоднозначным результатам VPN.

1.3 Анализ traceroute

1.3.1 Наблюдаемый маршрут

На стенде применены ICMP- и UDP-варианты:

```
traceroute -n -I -q 1 -w 1 -m 5 \
  172.28.22.10
traceroute -n -q 1 -w 1 -m 5 \
  172.28.22.10
```

В выводе обеих команд маршрут состоит из трёх переходов: R1 — 172.28.20.254, R2 — 172.28.21.20, сервер — 172.28.22.10. Запись stand-traceroute.pcap позволяет увидеть как исходные пробы, так и ответы. Для сопоставления подходят фильтры «icmp» и «udp || icmp». Первые ICMP-пробы и ответы сведены в таблицу 4. Реализация отправила также пробы с TTL 4 и 5: они уже были запущены до обработки ответа конечного узла. Поэтому число исходных проб в pcap превышает число строк итогового маршрута.

Таблица 4 — Первые три ICMP-пробы трассировки

Кадр пробы	TTL	Кто ответил	Результат
1	1	172.28.20.254	Кадр 2: ICMP 11/0
3	2	172.28.21.20	Кадр 4: ICMP 11/0
5	3	172.28.22.10	Кадр 6: ICMP 0/0

На рисунке 6 отобраны ответы двух маршрутизаторов и конечного узла. В кадре 2 раскрыты внешний ICMP Time Exceeded с Type=11, Code=0 и вложенная исходная Echo-проба с Type=8. Это позволяет отличить ошибку доставки от самого проверочного запроса.

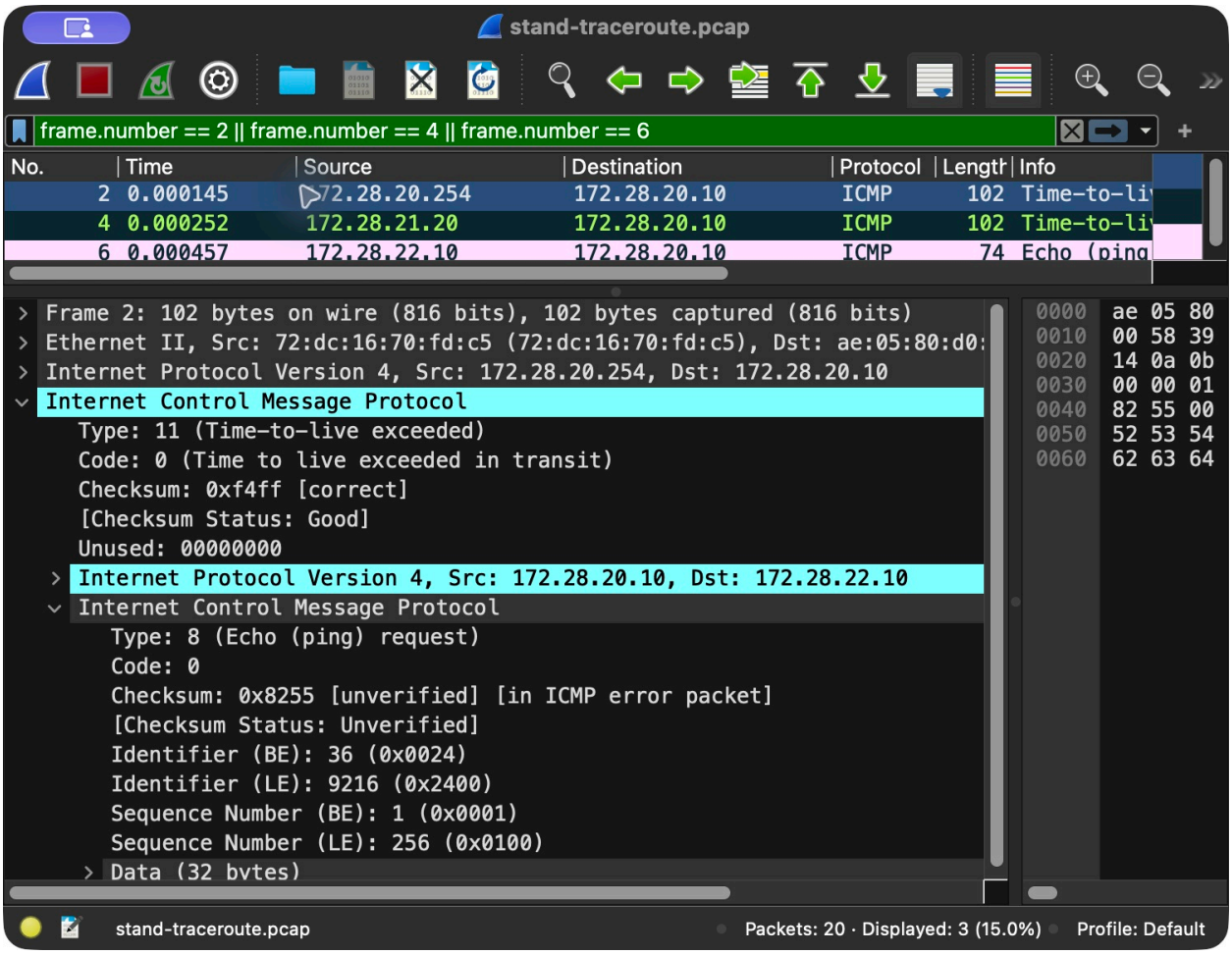


Рисунок 6 — ICMP-ответы маршрутизаторов и назначения в Wireshark

Каждый маршрутизатор уменьшает TTL. При исчерпании значения пакет не передаётся дальше, а источнику возвращается ICMP Time Exceeded. Постепенное увеличение начального TTL даёт ответы от последовательно более удалённых узлов. В конечном узле ICMP-вариант получает Echo Reply; UDP-вариант получает Destination Unreachable, Port Unreachable, поскольку целевой UDP-порт не обслуживается [2].

1.3.2 Размеры и различия сообщений

В кадре 1 заголовок IPv4 равен 20 байтам, Total Length — 60 байтам; поле данных IPv4 занимает 40 байт, включая восемь байт ICMP и 32 байта нагрузки Echo. В UDP-пробе кадра 11 Total Length также равен 60 байтам:

после заголовка IPv4 следуют UDP длиной 40 байт, из которых 8 байт — его заголовок. ICMP-ошибка кадра 2 имеет внешний IPv4 Total Length 88 байт и заголовок IPv4 длиной 20 байт. Следовательно, после внешнего IP-заголовка передаются 68 байт: заголовок ошибки ICMP и вложенная часть исходной пробы.

По вложенным адресам и параметрам ICMP либо UDP источник определяет, какой запрос вызвал ошибку. Echo Reply возвращает данные Echo, а ICMP error сообщает причину невозможности дальнейшей доставки; оба типа нужны для различения промежуточного ограничения и достижения цели. ICMP-пробы traceroute структурно похожи на ping: это Echo Request. Отличаются назначение серии, изменяемый TTL и выбранная длина данных. UDP-вариант traceroute формирует UDP-датаграммы, а ICMP появляется в его ответах. Утверждать, что traceroute всегда отправляет ICMP, некорректно.

Ключ Windows tracert «-d» и использованный в Linux/macOS «-n» отключают разрешение адресов узлов в имена. Без них возможны обратные DNS-запросы PTR; их наличие зависит и от кэша. Сохранённые пробы выполнены с отключённым разрешением: дополнительный опыт без ключа не проводился.

1.4 Анализ HTTP

1.4.1 Первичное получение страницы

На локальном сервере размещена учебная HTML-страница с ФИО студента. С помощью curl выполнены два запроса к http://172.28.22.10:8000/. Первый GET не содержит условия проверки версии. Второй передаёт дату Last-Modified в поле If-Modified-Since. Файл favicon.ico не запрашивался и в анализ не включён. В трассе stand-http-arp.pcap первый GET находится в кадре 7, ответ 200 OK — в кадре 11.

Поле Content-Length равно 476 байтам; полученное тело совпадает с исходным файлом. Заголовки ответа и тело переданы отдельными TCP-сегментами, поэтому размер одного сегмента не равен размеру всего HTTP-ответа. В кадре 11 полезная нагрузка TCP составляет 476 байт, а длина кадра — 542 байта: 14 байт Ethernet, 20 байт IPv4, 32 байта TCP с опциями и 476 байт файла. Состав запроса и сегмента с телом ответа изображён на рисунке

7. Направление TCP-портов меняется на обратное, а длина TCP-заголовка в обоих кадрах равна 32 байтам из-за опций.

Это авторская схема по полям TShark, а не снимок интерфейса анализатора.

Кадр 7: GET / HTTP/1.1; TCP 49298 ⟶ 8000

Ethernet II 14 байт	IPv4 20 байт	TCP 32 байта	HTTP GET 81 байт

Кадр 11: тело страницы; TCP 8000 ⟶ 49298

Ethernet II 14 байт	IPv4 20 байт	TCP 32 байта	Данные 476 байт

Рисунок 7 — Измеренные размеры блоков в HTTP-запросе и сегменте ответа

HTTP-запрос задаёт метод GET и путь «/», а Host указывает имя или адрес обслуживаемого ресурса. Ответ содержит строку состояния, служебные поля и при необходимости тело. Смысл этих частей сохраняется независимо от того, формирует запрос браузер или curl [4]. Дополнительно та же учебная страница открыта в новой вкладке Chrome. Трасса browser-http.pcap содержит первый запрос GET в кадре 5 и ответ 200 OK с Content-Length 476 в кадре 13.

После обычного обновления вкладки браузер отправил условный GET с If-Modified-Since (кадр 37); сервер ответил 304 Not Modified (кадр 39). Запрос favicon.ico и ответ 404 (кадры 19 и 23) не относятся к основной странице. Захват выполнен на loopback-интерфейсе только для порта учебного сервера. Значение локального Cookie в сохранённой копии замаскировано; длины пакетов сохранены.

1.4.2 Условный GET

Последовательность обмена приведена в таблице 5. Вторая команда явно добавила условный заголовок:

```
modified='Mon, 02 Nov 2026 14:26:33 GMT'
curl --noproxy '*' \
```

```
-H "If-Modified-Since: $modified" \
http://172.28.22.10:8000/
```

В кадре 19 действительно присутствует If-Modified-Since. Сервер установил, что файл не менялся после указанного момента, и ответил 304 Not Modified в кадре 21. Такой ответ не содержит тела представления; клиент может использовать ранее сохранённые 476 байт [4]. Экономия относится к повторной передаче содержимого, а не к исчезновению всех IP-, TCP- и HTTP-заголовков.

Таблица 5 — Обычный и условный HTTP GET на стенде

Кадр	**Сообщение**	**Смысл**
7	GET /	Первичное получение страницы
11	200 OK; 476 байт тела	Передача содержимого
19	GET / с If-Modified-Since	Проверка неизменности файла
21	304 Not Modified	Повторная передача тела не требуется

Развёрнутый запрос кадра 19 показан на рисунке 8. Поле If-Modified-Since содержит дату версии файла; анализатор связывает этот запрос с ответом в кадре 21.

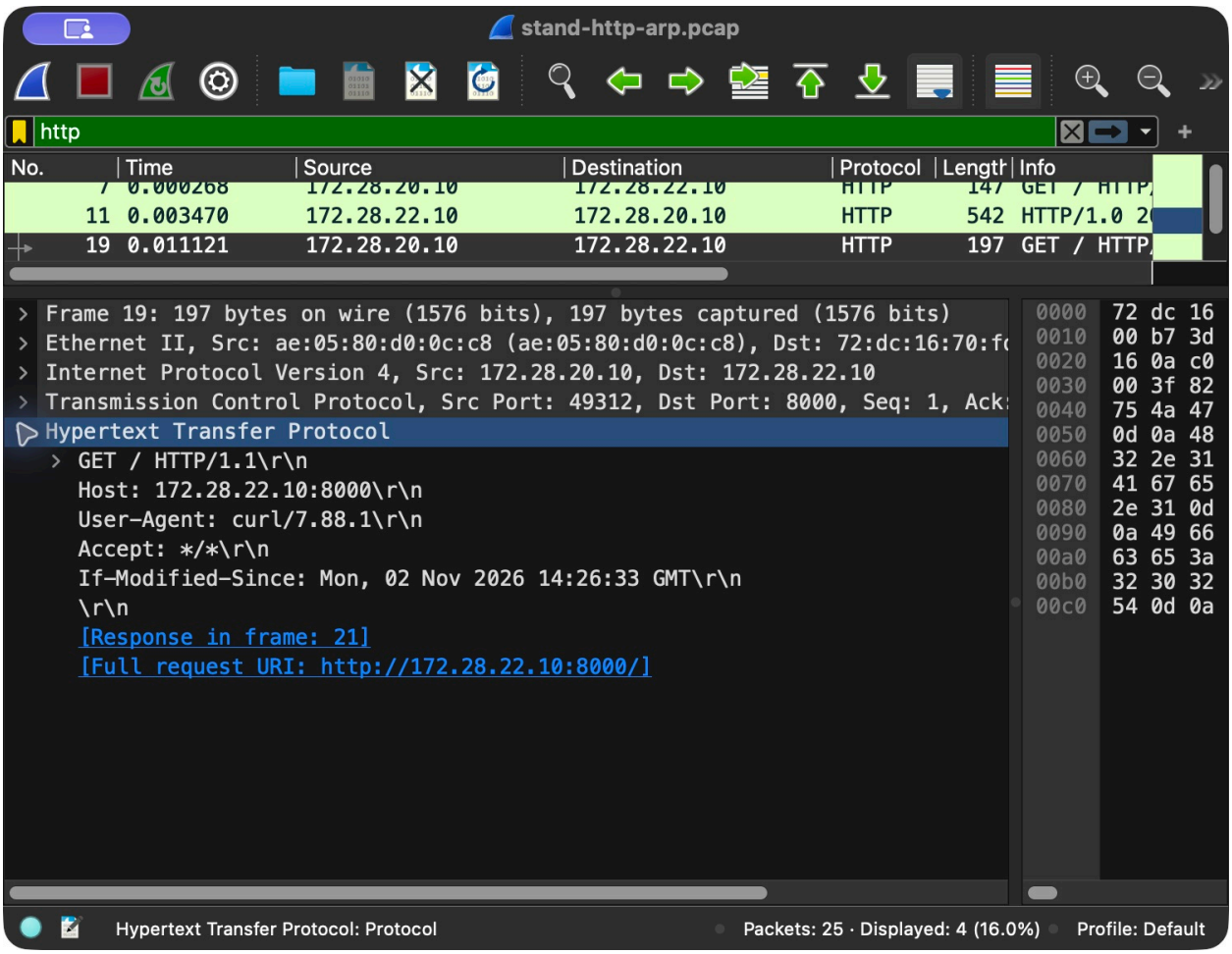


Рисунок 8 — Условный HTTP GET с полем If-Modified-Since в Wireshark

Фильтр «http» показывает прикладные сообщения; «tcp.port == 8000» позволяет также проследить установку, подтверждения и завершение TCP-соединений. На открытом HTTP заголовки читаемы в Wireshark. У HTTPS без ключей расшифрования наблюдаются TCP и TLS, но текст GET и содержимое личного кабинета недоступны.

1.5 Анализ DNS и nslookup

1.5.1 DNS-запрос браузера после очистки кэша

4 ноября 2026 года выполнен отдельный браузерный DNS-опыт. Системный кэш macOS очищен командой dscacheutil -flushcache (код завершения 0); в Chrome использована команда Clear host cache, затем корень ИСУ открыт в новом окне инкогнито. Условия сохранены в verification/browser-dns.json. На интерфейсе utun4 записан обмен с резолвером 172.19.0.2; в итоговой трассе оставлены только пакеты имени isu.ifmo.ru.

В browser-dns-isu.pcap находятся два пакета: A-запрос и соответствующий ответ с адресом 77.234.212.21. Совпадение Transaction ID и типа вопроса проверяется анализатором. Это самостоятельная запись браузерного опыта, отличная от внешних запусков nslookup. Поле Question указывает имя сайта, а адрес назначения IPv4 принадлежит резолверу. Отдельная загрузка изображений не исследовалась.

1.5.2 Запрос и ответ DNS

При внешнем nslookup имя isu.ifmo.ru запрашивалось у резолвера 172.19.0.2. В кадрах 54 и 55 трассы isu-utilities.pcap сохранены A-запрос и ответ с адресом 77.234.212.21. Адрес DNS-сервера отличается от адреса сайта, поскольку первый предоставляет службу разрешения имён, а второй принимает соединения приложения.

Резолвер может ответить из кэша или обратиться к другим DNS-серверам. Структура DNS разобрана по полной шестикадровой трассе stand-dns.pcap. Сообщение включает заголовок с идентификатором транзакции и флагами, Question, Answer, Authority и Additional. Вопрос задаёт имя, тип и класс, а ресурсные записи ответа содержат имя, тип, класс, TTL и данные. Идентификатор и повторённый вопрос позволяют сопоставить запрос с ответом [5]. Типы вопросов в реальном опыте приведены в таблице 6. Запрос A задаёт IPv4-адрес, AAAA — IPv6-адрес, NS — серверы зоны. Кроме них используются CNAME для псевдонимов, MX для почты, PTR для обратного разрешения, SOA для сведений о зоне и TXT для текстовых данных. Способ транспортировки, включая обычный DNS, DNS over TLS или DNS over HTTPS, не превращает A либо NS в другой тип ресурсной записи.

Таблица 6 — Вопросы nslookup и ответы DNS-сервера стенда

Кадры	**Тип**	**Наблюдаемый ответ**
1 → 2	A (1)	shibaev.test → 172.28.22.10; AA=1
3 → 4	AAAA (28)	NOERROR, Answers=0; запись IPv6 не задана
5 → 6	NS (2)	ns.shibaev.test; AA=1

В A-ответе кадра 2 одна запись в Answer и одна NS-запись в Authority. Заголовок ответа имеет AA=1: dnsmasq отвечает авторитативно для учебной зоны. Ответ AAAA без записей и с кодом NOERROR означает отсутствие

записи запрошенного типа, а не обязательное отсутствие самого имени. Размеры уровней для первой DNS-пары приведены на рисунке 9.

Общий ID 0x987a связывает вопрос и ответ. В ответе флаги равны 0x8500: QR=1, AA=1, RD=1, RA=0; сервер подтверждает свои полномочия для зоны, но не предлагает рекурсивное разрешение. Числа записей QD/AN/NS/AR равны 1/1/1/0.

Кадр 1: вопрос A; UDP 41176 → 53; ID 0x987a

Ethernet II 14 байт	IPv4 20 байт	UDP 8 байт	DNS 30 байт

Кадр 2: ответ A; UDP 53 → 41176; ID 0x987a

Ethernet II 14 байт	IPv4 20 байт	UDP 8 байт	DNS 87 байт

Рисунок 9 — Инкапсуляция запроса и авторитативного ответа DNS на стенде

Независимые DNS-запросы для изображений требуются, если их URL указывают на другие имена хостов, например на отдельный CDN, а подходящих записей ещё нет в кэше. Несколько изображений на одном уже разрешённом имени не требуют отдельного вопроса DNS для каждого файла. Этот вывод описывает механизм; отдельный опыт загрузки изображений браузером не проводился.

1.5.3 Сопоставление обычного nslookup и запроса NS

Выполнены команды:

```
nslookup shibaev.test 172.28.22.10
nslookup -type=NS shibaev.test 172.28.22.10
```

Первая команда данной версии nslookup создала A- и AAAA-запросы, вторая — NS-запрос. В сумме записаны три пары запросов и ответов. Указанные в старой методичке три пары для каждой команды не являются обязательным свойством всех реализаций nslookup: искусственно повторять или отбрасывать две первые пары здесь не требуется. Содержимое Answers определяется типом вопроса: в кадре 2 это IPv4-адрес, в кадре 6 — имя авторитативного сервера ns.shibaev.test.

Наличие AA относится к конкретному DNS-ответу; рекурсивный резолвер, переславший сохранённую NS-запись, сам от этого не становится авторитативным сервером зоны. Публичные DNS-запросы повторно записаны 3 ноября в отдельную трассу isu-dns-complete.pcap. В ней кадры 1–2 содержат A-запрос isu.ifmo.ru и ответ 77.234.212.21. Запрос NS того же имени в кадрах 3–4 не вернул NS-записей: Answers=0, в Authority находится SOA зоны ifmo.ru с первичным сервером ns.itmo.ru.

В кадрах 5–6 запрос NS к ifmo.ru вернул ns.ifmo.ru, ns2.ifmo.ru и ns3.ifmo.ru. Таким образом, отсутствие NS у имени хоста не означает отсутствие авторитативных серверов у содержащей его зоны. Флаг AA в ответах рекурсивного резолвера не установлен; авторитативный ответ непосредственно от DNS-сервера показан отдельно на стенде.

На рисунке 10 раскрыт A-ответ внешнего резолвера из кадра 2 этой трассы. В Answers указаны имя isu.ifmo.ru, тип A, класс IN и адрес 77.234.212.21.

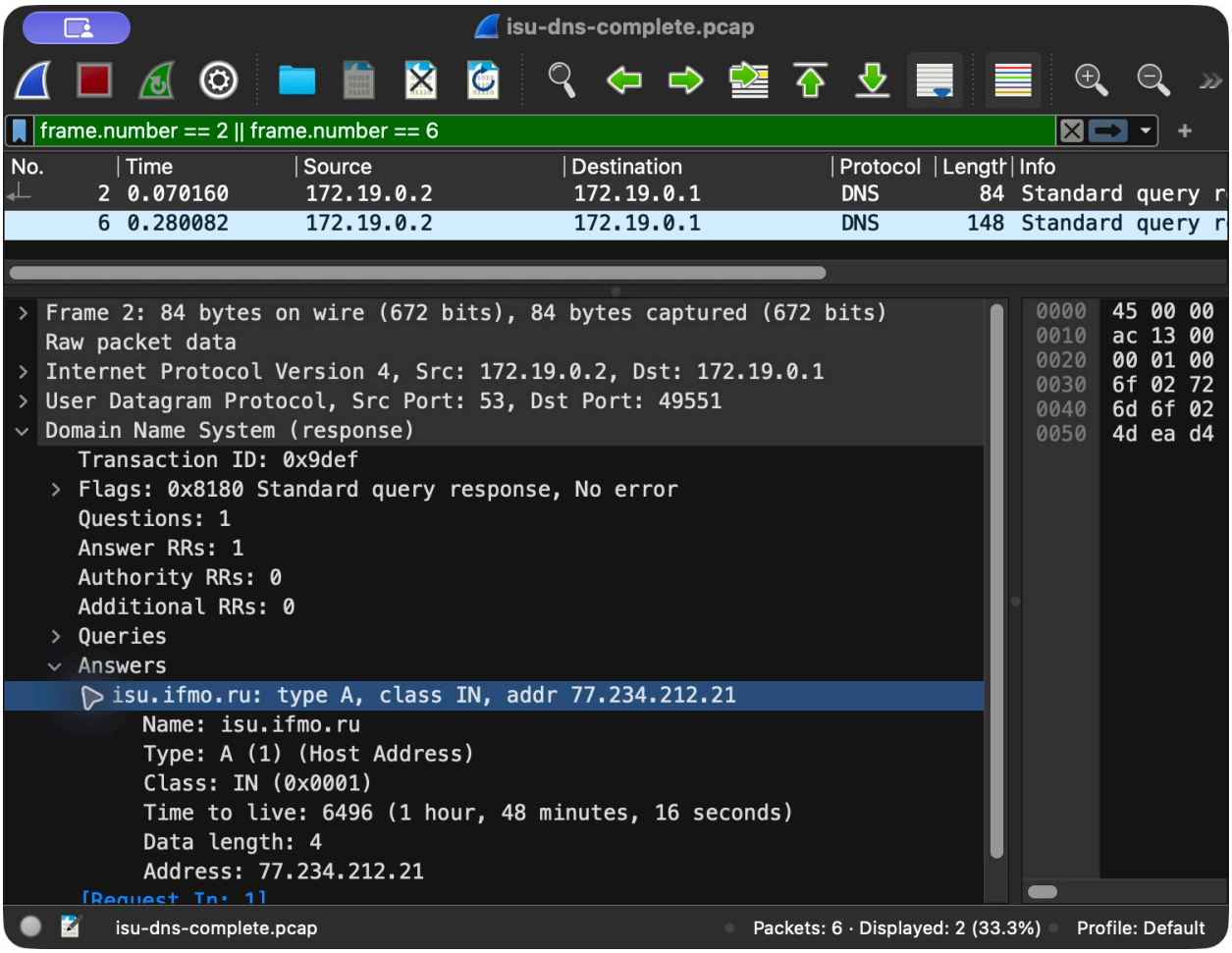


Рисунок 10 — Разрешение имени ИСУ в IPv4-адрес: внешний DNS-ответ

1.6 Анализ ARP

Перед первым HTTP-запросом очищена соседняя таблица только в клиентском контейнере:

```
ip neigh flush dev eth0
```

Клиенту требуется MAC-адрес следующего маршрутизатора 172.28.20.254, поскольку сервер 172.28.22.10 расположен за пределами локальной подсети. В кадрах 1 и 2 трассы stand-http-arp.pcap записан широковещательный ARP Request, в кадре 3 — ARP Reply. Наблюдаемые адреса приведены в таблице 7.

Таблица 7 — MAC-адреса в ARP и HTTP на первом сегменте

Сообщение	**Ethernet source**	**Ethernet destination**
ARP Request	ae:05:80:d0:0c:c8	ff:ff:ff:ff:ff:ff
ARP Reply	72:dc:16:70:fd:c5	ae:05:80:d0:0c:c8
HTTP GET у клиента	ae:05:80:d0:0c:c8	72:dc:16:70:fd:c5
HTTP-ответ у клиента	72:dc:16:70:fd:c5	ae:05:80:d0:0c:c8

Адрес ae:05:80:d0:0c:c8 принадлежит виртуальному интерфейсу клиента, 72:dc:16:70:fd:c5 — интерфейсу R1 в этом сегменте. FF:FF:FF:FF:FF:FF означает широковещательную доставку. Эти локально назначенные MAC-адреса не являются заводскими адресами физического Mac и не идентифицируют Интернет-сервер ИСУ. В HTTP GET конечный IPv4-получатель равен 172.28.22.10, а Ethernet-получатель — MAC маршрутизатора R1.

В HTTP-ответе на последнем переходе к клиенту Ethernet-источником снова выступает R1, хотя IPv4-источником является сервер. Это показывает различие канальной доставки и межсетевой адресации. ARP содержит тип аппаратной сети и протокола, длины адресов, код операции и адреса отправителя и цели. IP-адрес отправителя сообщает, кому нужно отвечать, и вместе с его MAC позволяет получателю обновить соответствие в соседней таблице. ARP передаётся непосредственно в Ethernet и не использует порты TCP или UDP [6].

Эти поля раскрыты в запросе кадра 2 на рисунке 11: искомый Target IP равен 172.28.20.254, а Target MAC пока заполнен нулями. Адрес получателя самого Ethernet-кадра при этом широковещательный.

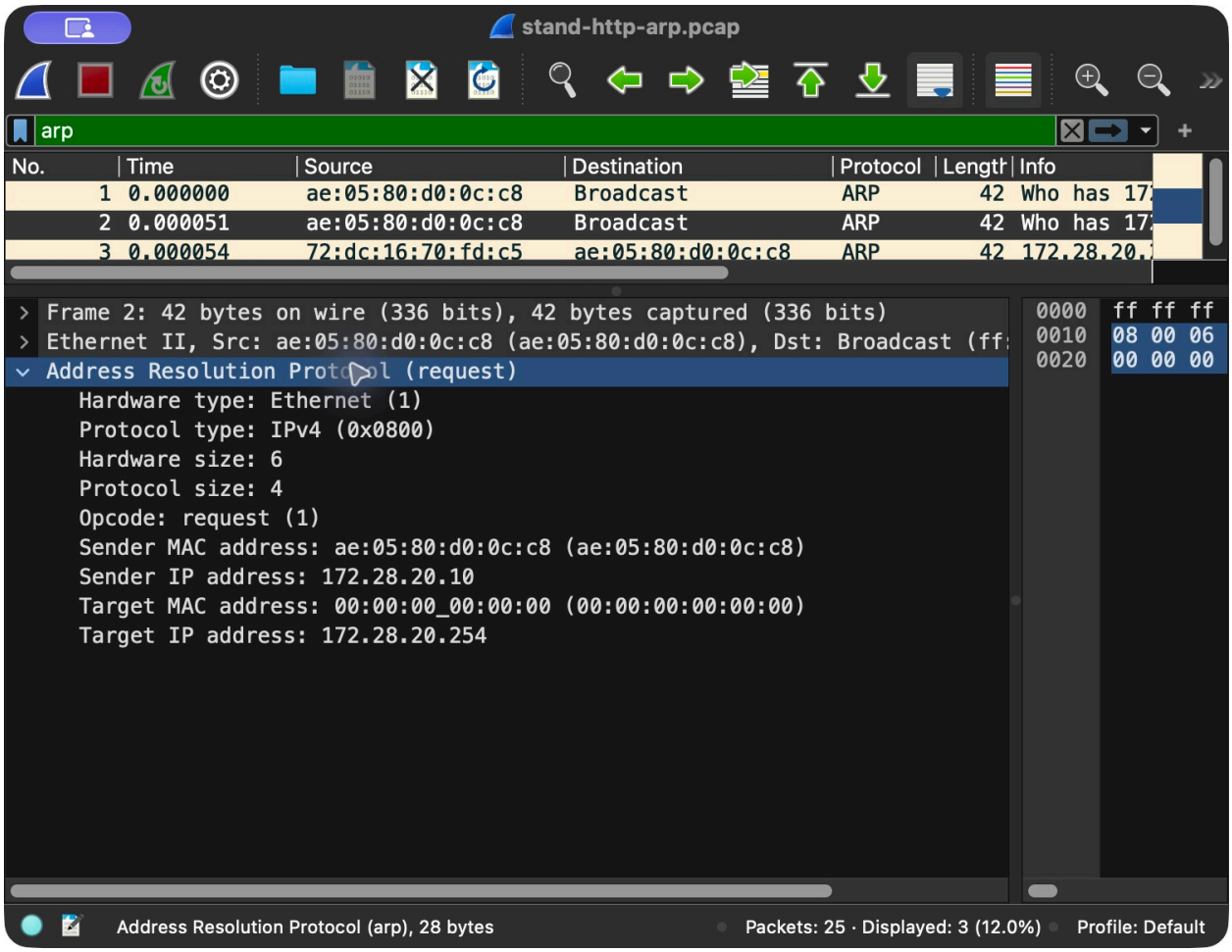


Рисунок 11 — Поля ARP-запроса MAC-адреса ближайшего маршрутизатора

1.7 Анализ FTP

1.7.1 Управляющее соединение и передача файла

Из учебного FTP-сервера загружен файл index.html. Опыт выполнен curl по URL ftp:

Открытый FTP передаёт команды и эти параметры без шифрования.

Таблица 8 — Основные сообщения FTP-сеанса

Кадр	Команда / код	Содержание
6	USER	anonymous
9	PASS	ftp@example.com

Кадр	Команда / код	Содержание
13–14	PASV / 227	Пассивный адрес 172.28.22.10:30005
18–19	TYPE I / 200	Двоичный режим передачи
20–21	SIZE / 213	Размер index.html — 476 байт
22–23	RETR / 125	Начало загрузки файла
24	FTP-DATA	476 байт содержимого
27	226	Передача завершена

В ответе на PASV сервер передал шесть чисел «172,28,22,10,117,53». Первые четыре задают IPv4-адрес, последние два — номер порта:

$$p = 117 \times 256 + 53 = 30005. \tag{2}$$

По формуле (2) клиент открыл соединение с локального порта 49780 к серверному порту 30005. Этот порт выбран сервером из настроенного диапазона 30000–30009; он не обязан быть равен 20. Порт 20 связан с классическим активным режимом, в котором соединение данных устанавливает сервер. Разделение управляющего и информационного каналов предусмотрено самим FTP [7].

Ответ 227 с аргументами пассивного режима показан на рисунке 12. В списке рядом отобраны пакет с содержимым файла и сообщение о завершении передачи.

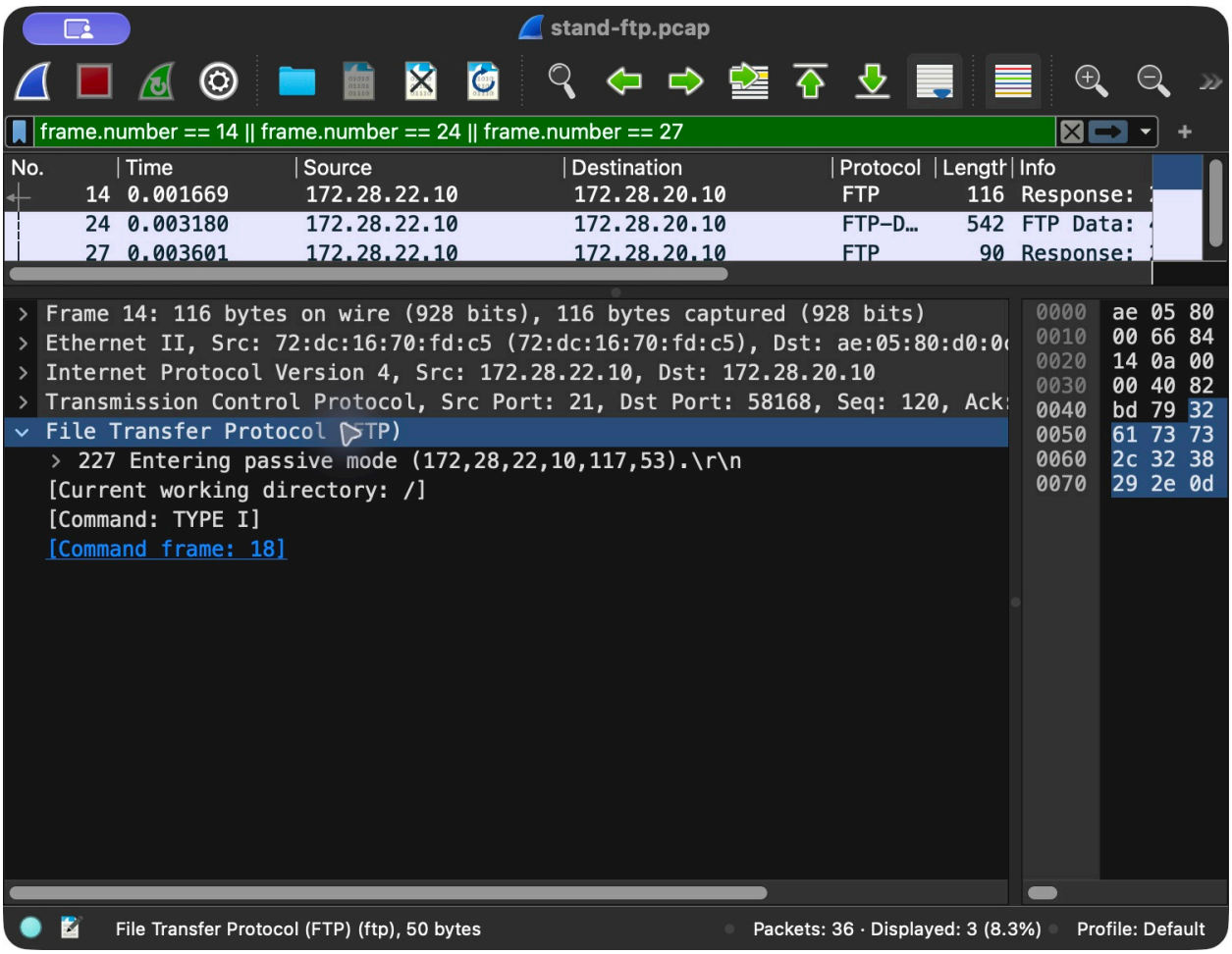


Рисунок 12 — Ответ PASV, информационный пакет и завершение передачи FTP

1.7.2 Содержание FTP-DATA

В кадре 24 полезная нагрузка TCP равна 476 байтам; это полный учебный HTML-файл. Проверено побайтовое совпадение скачанного файла с исходным. Значение 476 не является постоянным размером FTP-DATA: другой файл либо сегментация TCP могут дать несколько пакетов с другими длинами. Пакеты FTP на порту 21 содержат строковые команды и коды состояния, например RETR и 226. Поток FTP-DATA содержит сами передаваемые байты; он не повторяет управляющие команды.

В обоих случаях внешний порядок инкапсуляции одинаков — Ethernet, IPv4, TCP, но прикладное назначение и TCP-соединения различаются.

1.8 Анализ DHCP

1.8.1 Получение адреса и временная диаграмма

DHCP-опыт выполнен в отдельном контейнере клиента. Его начальный статический адрес Docker удалён; затем dhclient дважды получил и освободил аренду. Конфигурация сетевого интерфейса macOS не менялась. В обоих циклах сервер 172.28.23.2 предложил клиенту адрес 172.28.23.101. Трасса stand-dhcp.pcap содержит две завершённые транзакции DORA с идентификаторами 0x0b98fb01 и 0x7e91d065 и два DHCP Release.

В первом цикле логические фазы соответствуют кадрам 1, 6, 7 и 9. В захвате также присутствуют повторные Discover (кадр 2) и Request (кадр 8); они не означают дополнительных фаз выдачи адреса. На рисунке 13 показаны измеренные времена первой последовательности. Отсчёт ведётся от первого Discover. Направление стрелки задаёт направление сообщения, а подпись — UDP-порты.

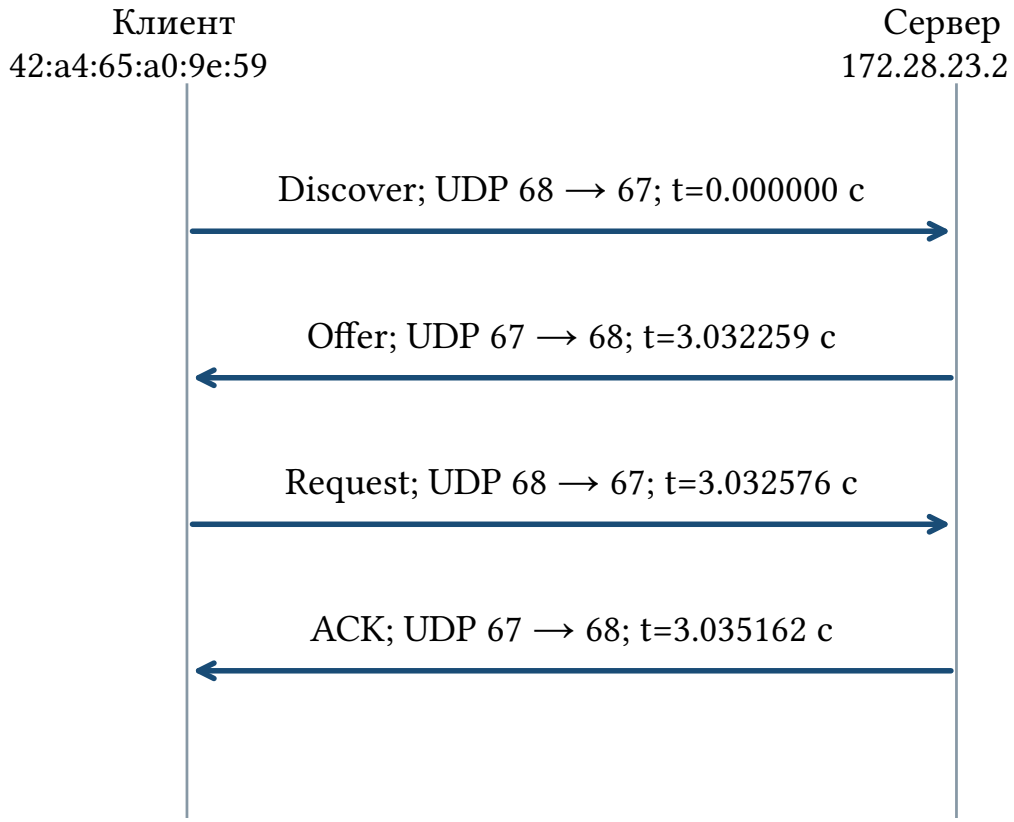


Рисунок 13 — Последовательность первой выдачи адреса DHCP

Discover предназначен для поиска серверов. Request в состоянии выбора подтверждает, какое предложение выбрано: в нём присутствуют

запрошенный адрес и идентификатор сервера. Offer предлагает параметры, ACK подтверждает их назначение. Общий формат наследует BOOTP и дополняется DHCP-опциями, среди которых тип сообщения и идентификатор сервера [8].

1.8.2 MAC- и IP-адреса

В Discover и Request клиент ещё не использует назначаемый адрес: IPv4-источник равен 0.0.0.0, получатель — 255.255.255.255. Ethernet-источник — MAC клиента 42:a4:65:a0:9e:59, получатель — FF:FF:FF:FF:FF:FF. Благодаря широковещательному запросу сервер обнаруживается до настройки IPv4 на клиенте. В наблюдаемых Offer и ACK сервер отправляет одноадресный ответ с IPv4-источником 172.28.23.2 и получателем 172.28.23.101.

На Ethernet-уровне используются MAC сервера 26:31:86:3a:a5:c5 и MAC клиента 42:a4:65:a0:9e:59. В поле yiaddr указано предложение 172.28.23.101. Поэтому нельзя утверждать, что все четыре DHCP-сообщения обязательно широковещательные: способ ответа зависит от состояния клиента, параметров запроса и возможностей сети.

После ACK команда ip addr показала адрес 172.28.23.101. Release передан уже от этого IPv4-адреса к серверу 172.28.23.2. Получение того же адреса во втором цикле не означает, что освобождение не произошло: сервер вправе повторно предложить доступную аренду тому же клиенту. Подтверждение аренды из кадра 9 раскрыто на рисунке 14.

В нём совпадают Transaction ID первой последовательности и адрес клиента 172.28.23.101 в поле Your (client) IP address. UDP-порты равны 67 ⟶ 68.

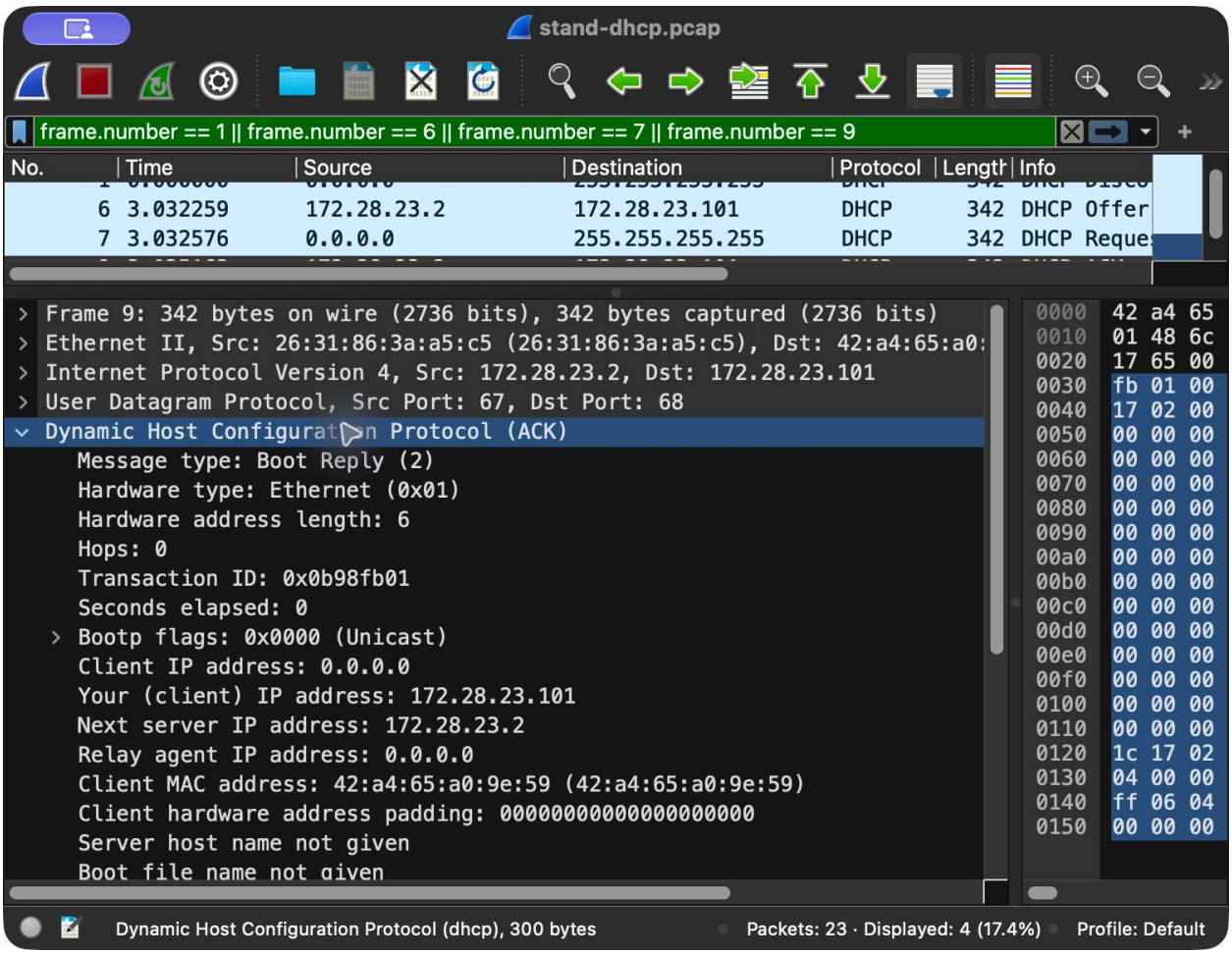


Рисунок 14 — DHCP ACK первой выдачи адреса 172.28.23.101

1.8.3 Фильтрация DHCP

В Wireshark 4.6.9 использован фильтр «dhcp». В старой методичке указан «bootp», поскольку DHCP использует формат BOOTP; современные имена полей приведены в справочнике Wireshark [9]. Независимый от имени диссектора вариант:

```
udp.port == 67 || udp.port == 68
```

Если очистить фильтр отображения, Wireshark покажет все пакеты, уже находящиеся в файле, включая ARP. Сам захват от этого не расширится задним числом: пакеты, исключённые фильтром захвата, в сохранённой трассе отсутствуют.

1.9 Анализ трафика Яндекс Телемоста

1.9.1 Условия опыта и сохранённые данные

Вместо Skype использован Яндекс Телемост, что допускается уточнениями к заданию. Сервис поддерживает звонки и чаты [10]. Тестовая встреча открывалась в двух локальных сеансах Chrome: организатор с учётной записью и гостевой участник в окне инкогнито. Внешние участники не приглашались. Камера и микрофон включались с разрешения владельца компьютера. Видеосеанс записан 2 ноября 2026 года в файл telemost-video.pcapng. Наблюдаемая длительность между первым и последним пакетами составляет 40,978843 с.

В трассе 21 632 пакета с суммарным исходным размером кадров 10 071 426 байт; средняя двунаправленная скорость по этим размерам равна 1966,171 кбит/с. Это сумма наблюдаемых потоков обоих локальных участников и служебного обмена, а не полезная скорость одного видеокодека. Захват ограничен UDP-обменом с медиасервером 77.88.42.102, адрес которого установлен по странице WebRTC Internals для тестовой встречи.

На интерфейсе Wi-Fi en0 записано 21 600 пакетов, в туннеле utun4 — 32 пакета. Прямой маршрут через Wi-Fi передавал основной медиатрафик, поэтому запись одного лишь туннеля оказалась недостаточной. Распределение размеров и интенсивности во времени показано на рисунке 15.

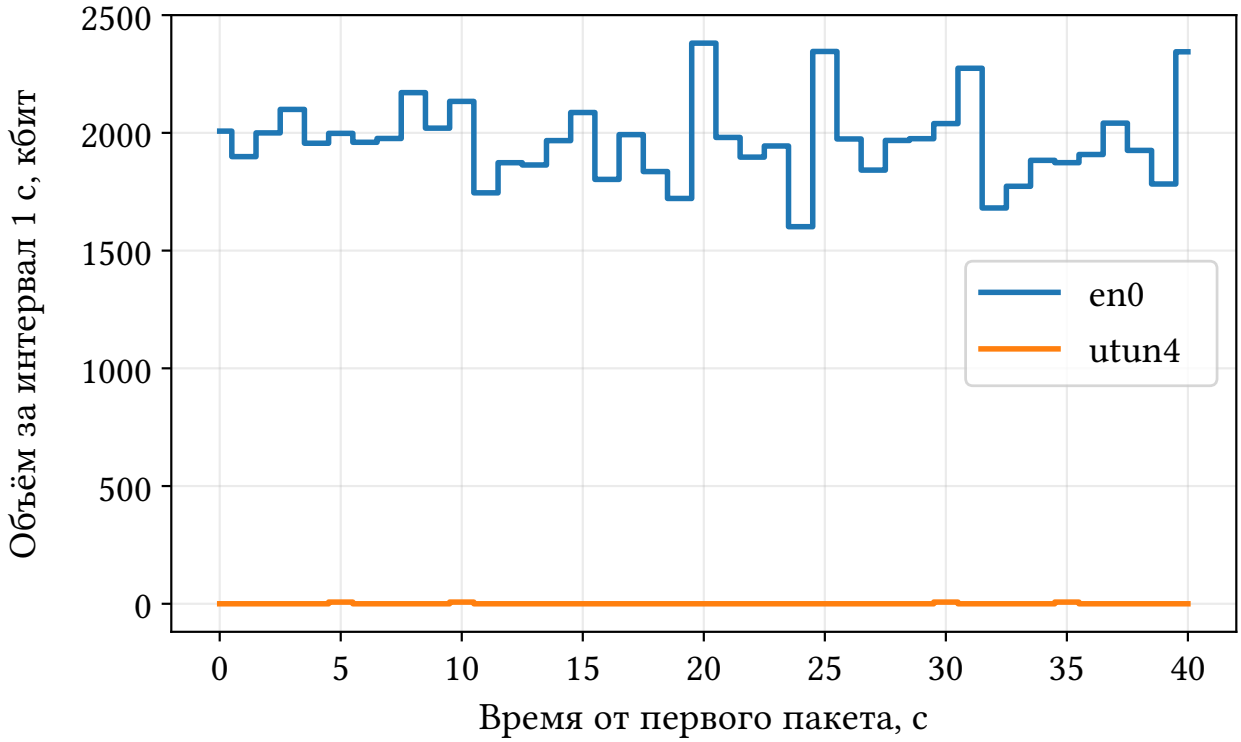


Рисунок 15 — Наблюдаемая интенсивность трафика тестового видеосеанса Телемоста

Размер сохраняемой части пакета ограничен 160 байтами; у 10 495 пакетов сохранено меньше исходной длины. Заголовки позволяют исследовать размеры, время, UDP-порты, SSRC и номера последовательности RTP. Полное содержимое кадров по такой трассе восстановить нельзя. Ключи TLS и SRTP не сохранялись, звук и изображение из сетевого дампа не расшифровывались.

Отдельный аудиосеанс с выключенными камерами повторно записан 3 ноября 2026 года в файл telemost-audio.pcapng. При восстановлении встречи выбран другой медиасервер — 77.88.42.81. В трассе 12 220 пакетов за 59,960023 с, суммарно 1 307 195 байт по исходным длинам кадров и 174,409 кбит/с. Основной обмен снова прошёл через en0: 12 188 пакетов; в utun4 записано 32 служебных пакета. Разделение потоков подтверждено состоянием устройств в приложении и статистикой WebRTC.

Текстовый опыт повторён 3 ноября. Два коротких контрольных сообщения последовательно отправлены из двух локальных вкладок одной учётной записи; их появление проверено в обеих вкладках. Состояние интерфейса сохранено в verification/telemost-text-ui-repeat.txt. Во время обмена записан выбранный WebSocket уведомлений чата: 172.19.0.1:58364

↔ 87.250.250.43:443. Его принадлежность Телемосту установлена по запросу к ws.push.yandex.ru со статусом 101 в DevTools и текущему TCP-сокету. Сеансовые параметры URL в комплект работы не включены.

В telemost-text.pcapng находятся 59 пакетов за наблюдаемый интервал 44,952695 с: 21 017 байт по исходным длинам IP-пакетов и 3,740 кбит/с. Вся запись получена на utun4, 28 пакетов направлены от клиента и 31 — к клиенту. Захват начат после установки TLS-соединения; сам HTTP-переход в WebSocket подтверждён интерфейсом DevTools. Сохранённая часть каждого пакета ограничена 160 байтами; 28 пакетов усечены. Значения frame.len и временные метки сохранены.

На рисунках 16 и 17 сопоставлены текстовый TLS-пакет и UDP-пакет отдельного аудиосеанса. В первом случае видны Raw IP, TCP и TLS, во втором — Ethernet, IPv4 и UDP. Надпись об усечении относится к сохранённой части пакета и не означает потерю его байтов в сети.

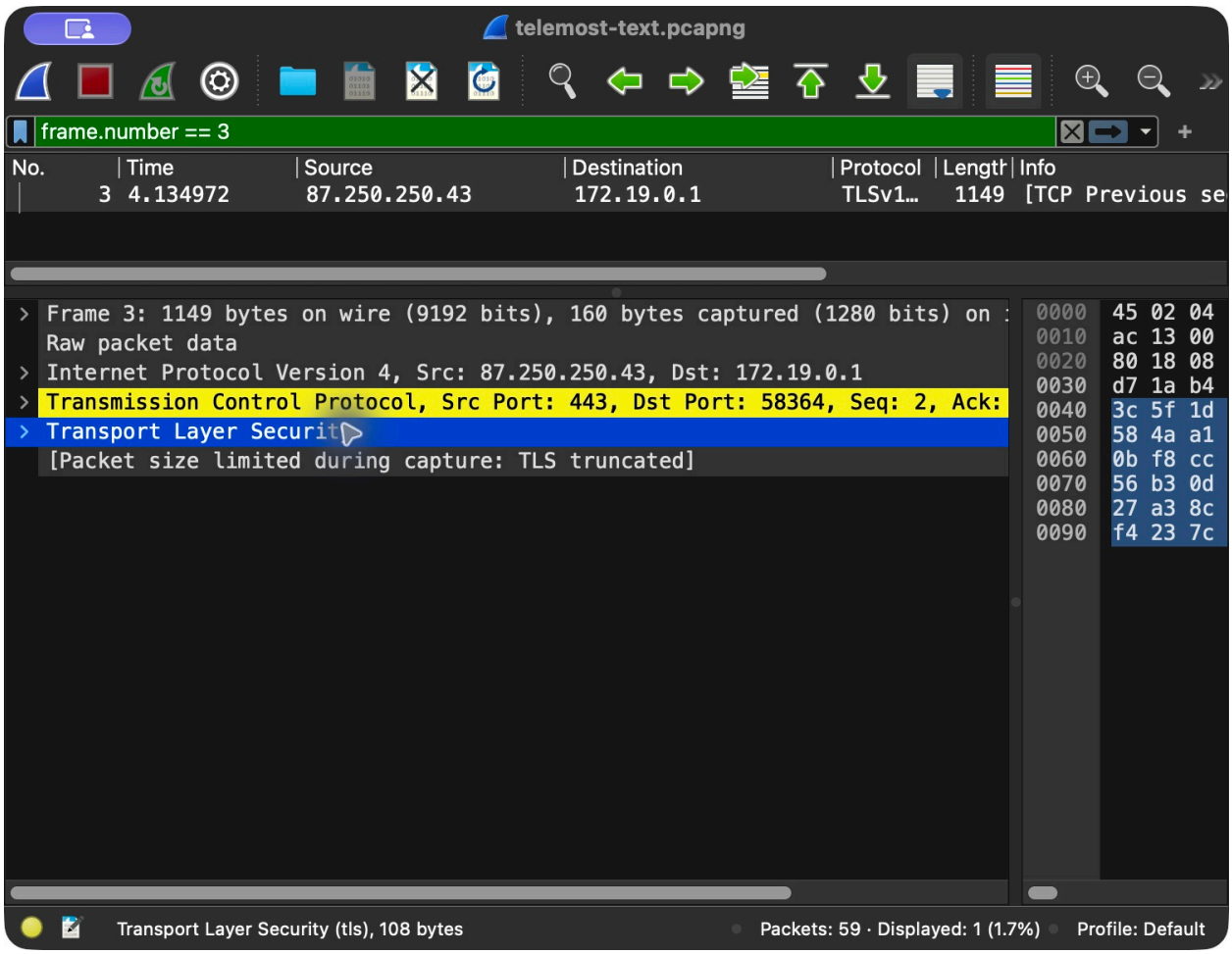


Рисунок 16 — Текстовый обмен: кадр 3 трассы telemost-text.pcapng, Raw IP → TCP → TLS

Аудиопакет на рисунке 17 передан другому серверу и использует UDP. Его исходная длина равна 114 байтам; UDP-длина составляет 80 байт, включая заголовок 8 байт и нагрузку 72 байта. Эти величины характеризуют выбранный пакет самостоятельного аудиосеанса.

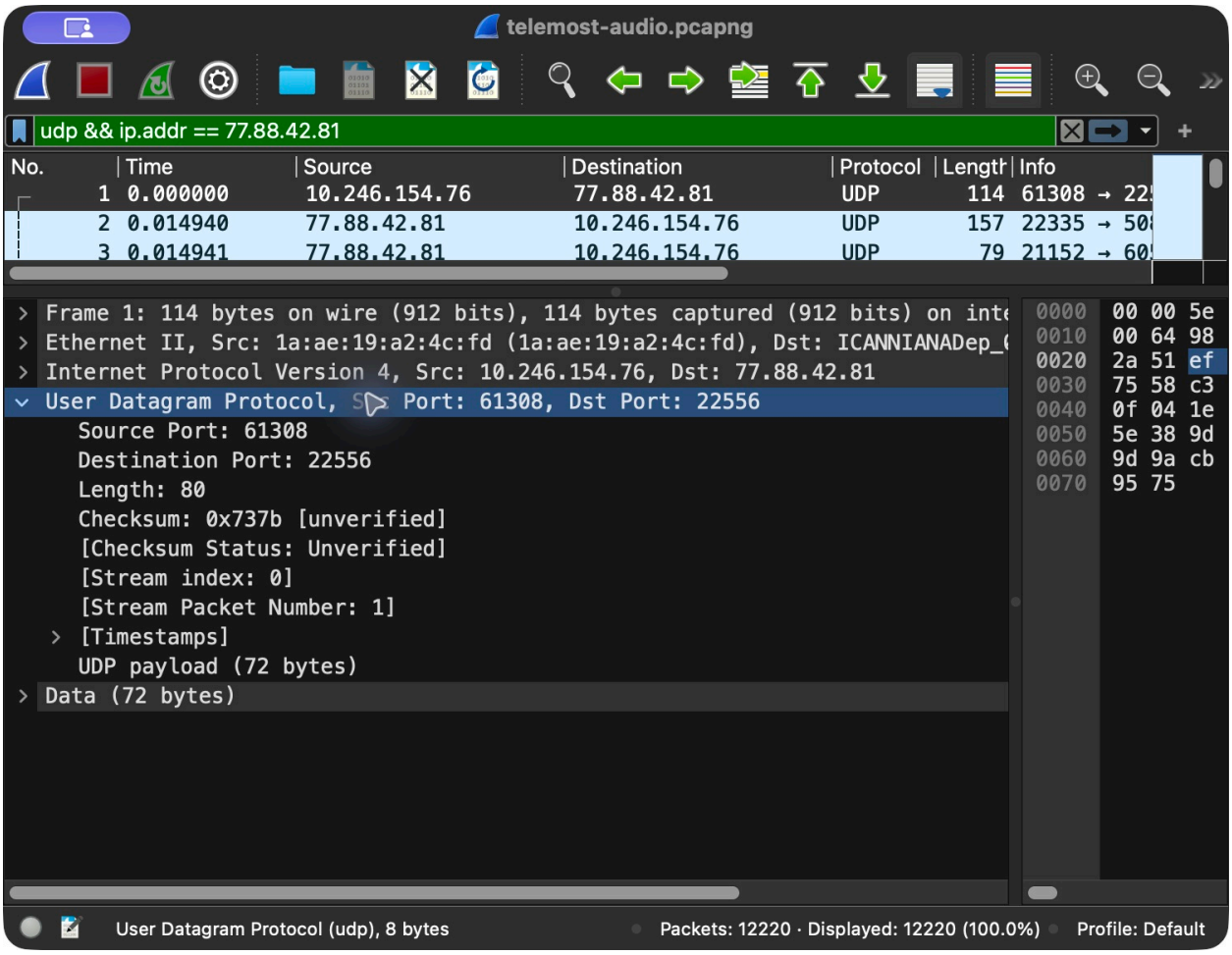


Рисунок 17 — Аудиообмен: кадр 1 трассы telemost-audio.pcapng, Ethernet → IPv4 → UDP

В таблице 9 и на рисунке 18 сопоставлены три записи. Текстовая запись охватывает выбранный канал уведомлений и служебный обмен, а аудио и видео — медиапотоки двух локальных участников. У текста frame.len относится к Raw IP, у основной медиазаписи — к Ethernet. Поэтому таблица характеризует конкретные захваты с разным охватом. Видеосеанс включает аудио; его общий трафик не равен скорости одного видеокодека.

Таблица 9 — Измеренные характеристики выбранных захватов Телемоста

Этап	**Длительность, с**	**Пакетов**	**кбит/с**
Текст (WebSocket)	44,952695	59	3,740
Аудио	59,960023	12 220	174,409

Этап	Длительность, с	Пакетов	кбит/с
Видео с аудио	40,978843	21 632	1966,171

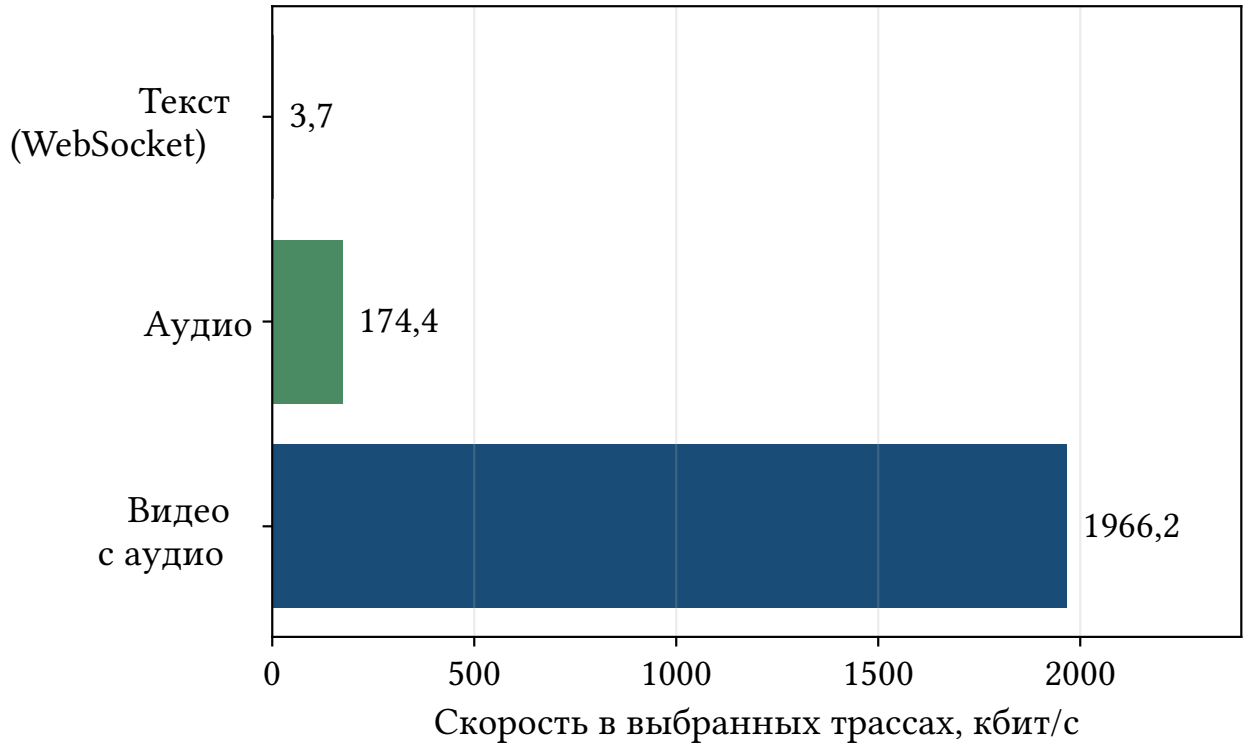


Рисунок 18 — Средняя двунаправленная интенсивность трёх выбранных захватов

Предварительная пустая попытка записи текста сохранена в verification/attempts/telemost-text-empty.pcapng. В результатах использован отдельный повторный опыт с проверенным транспортом чата. Полезная нагрузка TLS и SRTP не расшифровывалась.

1.9.2 Структура пакетов и классификация потоков

Для Wi-Fi-интерфейса порядок инкапсуляции исследуемого медиапакета имеет вид: Ethernet → IPv4 → UDP → защищённый RTP. Ethernet адресует ближайший узел канального сегмента, IPv4 обеспечивает доставку между IP-узлами, UDP различает транспортные конечные точки. Доступный RTP-заголовок содержит версию, тип полезной нагрузки, номер последовательности, временную метку и SSRC.

Эти поля описывают медиапоток, но не открывают защищённое содержимое. В WebRTC также используются ICE/STUN для проверки связности и DTLS для согласования защиты [11]. При анализе сохранённой

трассы к UDP-портам 21473, 22207, 22950 и 22759 применено Decode As → RTP. Это принудительная интерпретация структуры открытого заголовка; она не означает расшифрование SRTP.

В отбор включены пакеты с версией RTP 2. Измерения отдельных потоков представлены в таблице 10 и на рисунке 20. Пример разбора кадра 4 видеосеанса показан на рисунке 19. В GUI RTP назначен соответствующему локальному UDP-порту 62155. Открытый заголовок содержит PT 98, последовательный номер 31333, временную метку 2628388026 и SSRC 0x2e7bcf73.

В кадре сохранены первые 160 из 1044 исходных байт.

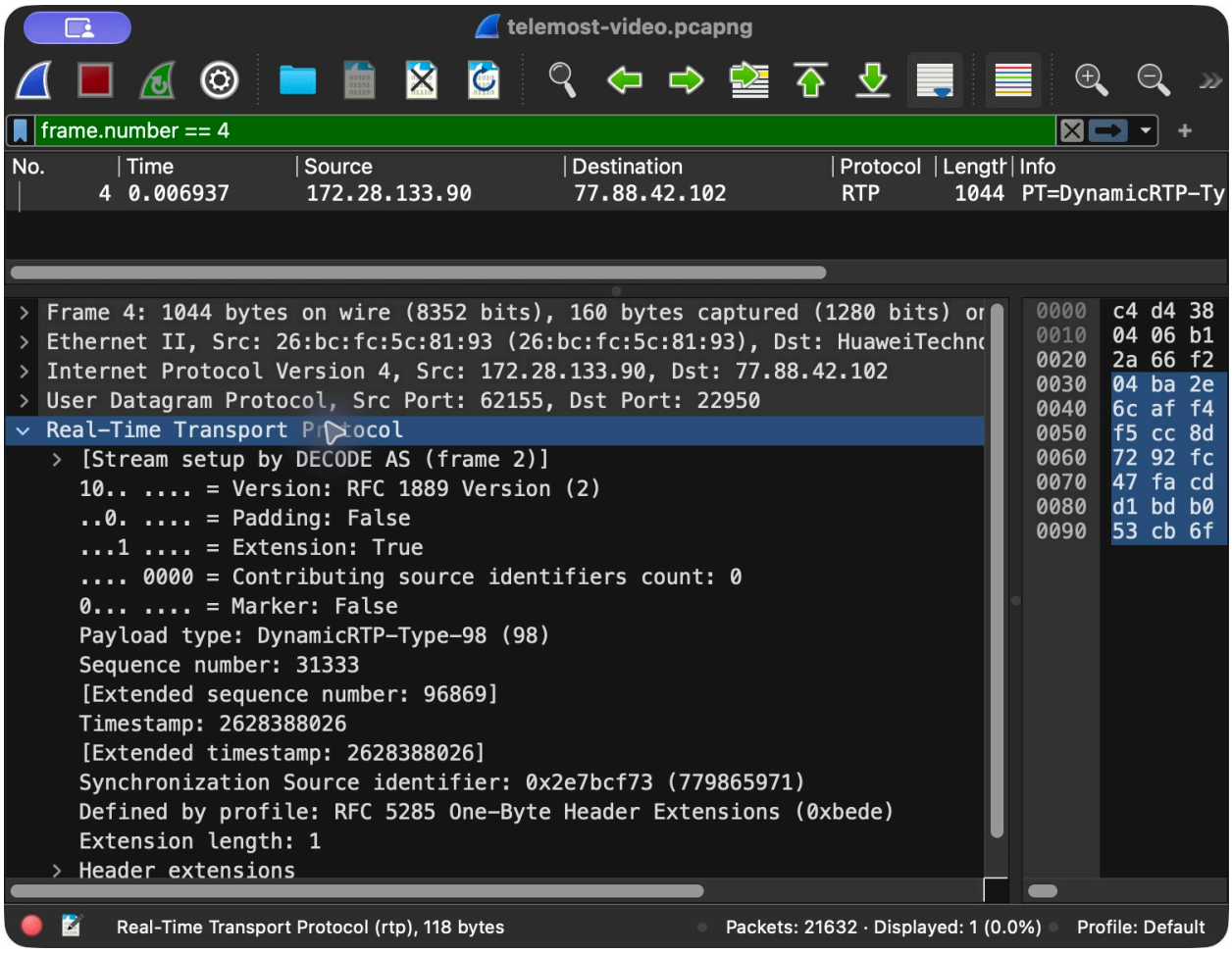


Рисунок 19 — Поля RTP-заголовка видеосеанса после Decode As

Таблица 10 — Примеры потоков одного сеанса по открытым RTP-заголовкам

SSRC	PT	Пакетов	Средний кадр, байт
0x6fb51938	111	2049	79,000
0x2e7bcf73	98	5838	952,492

SSRC	PT	Пакетов	Средний кадр, байт
0x6444244d	98	5819	356,921

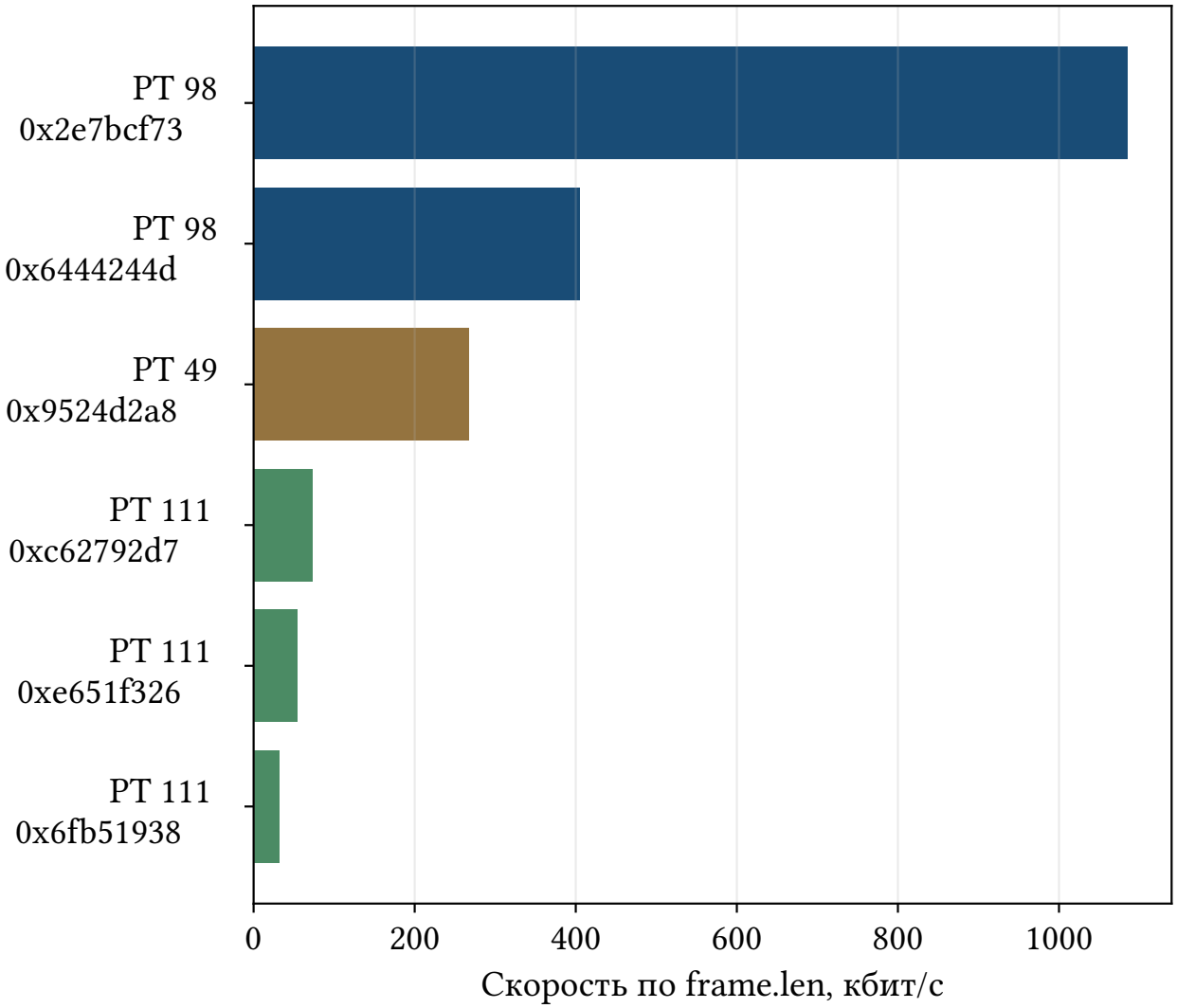


Рисунок 20 — Сравнение размеров кадров и интенсивности потоков одного сеанса

Для потока SSRC 0x6fb51938, поступающего с UDP-порта 21473, ранее наблюдавшаяся статистика WebRTC Internals указывала audio/opus и PT 111. В сохранённой трассе найдено 2049 соответствующих пакетов, медиана межпакетного интервала равна 20,0336 мс, средний размер кадра — 79 байт. Это реальный аудиопоток внутри видеосеанса.

В самостоятельном аудиосеансе гостевой клиент принимал Opus PT 111 с SSRC 0xeec9d003, что подтверждено его WebRTC-статистикой. В трассе найдено 2998 пакетов этого потока, медиана межпакетного интервала — 19,999 мс. Всего в аудиосеансе выделены 8995 пакетов с открытыми RTP-

заголовками и три SSRC с PT 111. Для их разбора использованы порты текущего сеанса 22556 и 22335; применять к нему старые порты видеосеанса было бы неверно. Потоки с PT 98 наблюдались при включённой камере. Они имеют больше пакетов, более крупные кадры и группы пакетов с одинаковой RTP-временной меткой.

Такое поведение согласуется с разбиением видеокадра на несколько пакетов. Приписывать PT 98 конкретный кодек по одному номеру нельзя: динамические значения PT определяются согласованием сеанса. Точное имя видеокодека здесь не устанавливалось. Поток с PT 49 оставлен неклассифицированным; сам по себе размер или тип PT не доказывает назначение потока.

1.9.3 Ответы на вопросы задания

Различие текста, аудио и видео. В выбранном текстовом канале наблюдается TCP/TLS: подтверждения TCP и зашифрованные прикладные записи. В захвате utun4 инкапсуляция начинается с IPv4 → TCP → TLS, заголовка Ethernet нет. Число и размер TLS-пакетов не дают длину исходного текстового сообщения: в соединении передаются также уведомления и служебные данные. Самостоятельный аудиосеанс содержит RTP-пакеты с медианным интервалом около 20 мс.

При включённой камере наблюдаются PT 98, более крупные RTP-пакеты и более высокая общая интенсивность, согласующиеся с видеопередачей. Привязка потоков к действиям опирается на состояние приложения и параметры сеанса.

Фильтры. Отбор пакетов выполняется фильтрами отображения Wireshark [12]. Первый фильтр ниже выделяет медиасервер видеосеанса, второй — аудиосеанса, третий — проверенный текстовый сокет. Адреса получены для конкретных подключений и при повторном опыте должны быть проверены заново.

```
ip.addr == 77.88.42.102 && udp
ip.addr == 77.88.42.81 && udp
ip.addr == 87.250.250.43 && tcp.port == 58364
```

После Decode As для исследованных портов аудиопоток из таблицы 10 выделяется по SSRC, а наблюдённые потоки PT 98 — по их типу полезной нагрузки:

```
rtp.ssrc == 0x6fb51938
rtp.p_type == 98 && rtp.version == 2
```

Универсального порта для разделения видов трафика нет: адреса и порты зависят от подключения. Здесь текст выделен по проверенному TCP-сокету, аудио и видео — по медиасерверу и полям RTP после Decode As. TLS шифрует содержимое, поэтому конкретную фразу нельзя сопоставить с отдельным пакетом по одним сетевым заголовкам.

ЗАКЛЮЧЕНИЕ

По сохранённым трассам исследованы заголовки и обмены ICMP, IPv4, TCP, UDP, HTTP, DNS, ARP, FTP и DHCP. На управляемом маршруте с двумя промежуточными маршрутизаторами показаны уменьшение TTL, сообщения Time Exceeded и достижение конечного узла. Для 13 размеров данных ICMP от 100 до 10 000 байт получен график числа IPv4-фрагментов; при MTU 1500 байт переход от одного пакета к двум происходит после 1472 байт данных ICMP.

В паре HTTP-запросов получены ответы 200 с телом 476 байт и 304 без повторной передачи тела. Показано, что MAC-адрес назначения кадра в другой подсети принадлежит ближайшему маршрутизатору. Для DNS исследованы различие запросов A и NS, авторитативный ответ учебного сервера и отсутствие NS-записи у имени хоста ИСУ при наличии NS-записей его зоны.

В FTP наблюдались отдельные управляющее и информационное соединения, в DHCP — два цикла выделения и освобождения адреса. Внешние измерения ИСУ и локальные опыты стенда разделены: ограничения доступного пути, ошибка отправки больших ping и ответ 403 на неавторизованный HTTPS-запрос не подменяются результатами локального сервера.

Для Телемоста сохранены самостоятельные аудио- и видеосеансы; открытые RTP-заголовки сопоставлены с состоянием приложения. Текстовый опыт подтвердил доставку двух контрольных сообщений между локальными вкладками одной учётной записи. Его TLS-захват ограничен выбранным соединением уведомлений, поэтому не описывает весь сетевой объём чата. Содержимое сообщений и медиаданных не расшифровывалось. Числовые таблицы и графики воспроизводятся из pcap и pcapng; исходники стенда, журналы команд, проверки и редактируемый исходник отчёта сохранены вместе с работой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алиев Т. И., Соснин В. В., Шинкарук Д. Н. Компьютерные сети и телекоммуникации: задания и тесты [Электронный ресурс]. Санкт-Петербург: Университет ИТМО, 2018. 112 с. URL: https://books.ifmo.ru/file/pdf/2275.pdf (дата обращения: 03.11.2026).

2. Postel J. Internet Control Message Protocol : RFC 792 [Электронный ресурс]. 1981. URL: https://rfc-editor.org/rfc/rfc792 (дата обращения: 03.11.2026).

3. Postel J. Internet Protocol : RFC 791 [Электронный ресурс]. 1981. URL: https://rfc-editor.org/rfc/rfc791 (дата обращения: 03.11.2026).

4. Fielding R., Nottingham M., Reschke J. HTTP Semantics : RFC 9110 [Электронный ресурс]. 2022. URL: https://rfc-editor.org/rfc/rfc9110 (дата обращения: 03.11.2026).

5. Mockapetris P. Domain Names — Implementation and Specification : RFC 1035 [Электронный ресурс]. 1987. URL: https://rfc-editor.org/rfc/rfc1035 (дата обращения: 03.11.2026).

6. Plummer D. C. An Ethernet Address Resolution Protocol : RFC 826 [Электронный ресурс]. 1982. URL: https://rfc-editor.org/rfc/rfc826 (дата обращения: 03.11.2026).

7. Postel J., Reynolds J. File Transfer Protocol : RFC 959 [Электронный ресурс]. 1985. URL: https://rfc-editor.org/rfc/rfc959 (дата обращения: 03.11.2026).

8. Droms R. Dynamic Host Configuration Protocol : RFC 2131 [Электронный ресурс]. 1997. URL: https://rfc-editor.org/rfc/rfc2131 (дата обращения: 03.11.2026).

9. Wireshark Foundation. Display Filter Reference: Dynamic Host Configuration Protocol [Электронный ресурс]. URL: https://www.wireshark.org/docs/dfref/d/dhcp.html (дата обращения: 03.11.2026).

10. Яндекс. О Яндекс Телемосте [Электронный ресурс]. URL: https://yandex.ru/support/yandex-360/customers/telemost/web/ru/ (дата обращения: 03.11.2026).

11. Alvestrand H. Transports for WebRTC : RFC 8835 [Электронный ресурс]. 2021. URL: https://rfc-editor.org/rfc/rfc8835 (дата обращения: 03.11.2026).

12. Wireshark Foundation. Wireshark User's Guide [Электронный ресурс]. URL: https://www.wireshark.org/docs/wsug_html_chunked/ (дата обращения: 03.11.2026).